

Juniorufgabe 1: Bällebad

Team-ID: 01004

Team: Wawa

Bearbeiter dieser Aufgabe:
Raphael Porsche

26. August 2025

Inhaltsverzeichnis

Programm-Ausführung.....	1
Lösungsidee.....	1
Umsetzung.....	1
Werkzeuge.....	2
Beispiele.....	2
Quellcode.....	3

Programm-Ausführung

Programmiersprache/Interpreter: Julia 1.12

Getestet auf: Linux, Windows

Packages: Keine

Installations und Benutzungsanweisung: Anweisung auf <https://julialang.org/install/> folgen um Julia zu installieren. Öffnen Sie ein Terminal und bewegen Sie das Arbeitsverzeichnis auf das src Verzeichnis in Junioraufgabe1. Verwenden sie diesen Befehl um das Programm auszuführen: „julia Junioraufgabe1.jl“. Die Ausgaben werden im Terminal zurückgegeben sowohl auch in ausgaben.txt im Junioraufgabe1 Verzeichnis gespeichert. Weiter Beispiele können in das „eingaben“ Verzeichnis gelegt werden.

Lösungsidee

Um die Zeit zu finden, in der die meisten Medizinbälle benötigt sind, müssen wir die Überlappungen der verschiedenen Zeitspannen finden und ihre Ballzahlen richtig zusammen addieren. Nur dann können wir wissen, zu welchem Zeitpunkt die meisten Bälle benötigt sind.

Aber dadurch, dass diese Zeitspannen immer an vollen Stunden beginnen und enden, können sich auch die Überlappungen nur zu vollen Stunden ergeben. Folgendermaßen reicht es aus wenn wir für jede der 24 vollen Stunden jeden Tages herausfinden, welche Zeitspannen dort stattfinden und deren

benötigten Medizinbälle addieren, um zu wissen wie viele Bälle wann benötigt sind. So arbeiten wir nicht mehr mit durchgehenden Zeitbereichen, sondern eine begrenzte Zahl von vollen Stunden.

Um dann die maximale Anzahl zu finden, müssen wir für jede Stunde jedes Tages die hinterlegte Ballanzahl vergleichen.

Umsetzung

(Q1 – Q2 referenziert verschiedene Teile des Quellcodes)

Um die verschiedenen Zeitspannen miteinander zu addieren, erstellen wir zunächst eine Hashmap, die die verschiedenen Tage zu einem Vektor von Zahlen zuordnet. Dieser Vektor repräsentiert die benötigten Bälle für jede Stunde. Hierbei sind die Indizes die verschiedenen Uhrzeiten (z.B. am Index 12, die Stunde von 11 zu 12) und der jeweilige Wert die Anzahl an benötigten Bällen. Jeder Vektor hat also auch nur 24 Einträge. (Q1)

Daraufhin gehen wir nun durch jede Zeile der Eingabe. Wenn der Tag dieser Zeile noch nicht in der Hashmap vorhanden ist, erstellen wir ein neues Element in der Hashmap, dass nun den neuen Tag zu einem Vektor von 24 Nullen zuordnet. Anschließend durchlaufen wir jede ganze Stunde der angegebenen Sportstunden Zeitspanne, und addieren die benötigte Anzahl an Bällen in der jeweiligen Stunde dieses Tages. (Q1)

Zum Schluss müssen wir nun durch die ganze Hashmap gehen und den Maximalwert finden. Dafür merken wir uns immer den bisherig höchsten Wert sowohl auch an welchem Tag zu welcher Stunde dieser passiert. (Julia ist 1 indexiert, also steht das erste Element am Index 1 für die erste Stunde des Tages von 0 – 1 Uhr.) (Q2)

Werkzeuge

Keine.

Beispiele

Beispiel	Ausgabe (Tag, Uhrzeit (Start der Stunde), Anzahl)
ball00.txt	("Montag", 14, 60)
ball01.txt	("Mittwoch", 11, 96)
ball02.txt	("Montag", 10, 157)
ball03.txt	("Freitag", 9, 152)
ball04.txt	("Montag", 10, 31)
ball05.txt	("Donnerstag", 8, 60)

ball06.txt	("Dienstag", 9, 90)
ball07.txt	("Freitag", 10, 59)

Quellcode

HashMap einlesen (Q1):

```
days = Dict{String, Vector{Int}}{()
for (idx, data) in enumerate(input)
    if !haskey(days, data[2])
        days[data[2]] = zeros{Int,24}
    end

    for x in (numbers[idx][1] + 1):numbers[idx][2]
        days[data[2]][x] += numbers[idx][3]
    end
end
```

(Bei zwei Integer x und y, erstellt x:y ein iterierbares Objekt, dass jeden Integer von x zu y beinhaltet, inklusive x und y. 'numbers' ist ein Vektor für jede Eingabezeile, in dem jedes Element die 3 letzten Zahlen der Zeile speichert.)

Maximalpunkt finden (Q2):

```
highest, highestidx = 0, 0
highestday = ""

for (day, content) in days
    for (hour, balls) in enumerate(content)
        if balls > highest
            highest = balls
            highestidx = hour - 1
            highestday = day
        end
    end
end
```